Day 3 — Python Object-Oriented Programming (OOP)

Study Notes, Practice Programs, and Student Management Console Application

Learning objectives: Understand classes and objects, constructors, encapsulation, inheritance, polymorphism, abstraction, access conventions, methods, and method overriding. Build Student and Employee classes and a menu-driven Student Management application.

1. What Is Object-Oriented Programming?

Object-Oriented Programming (OOP) is a programming approach that organizes code using classes and objects. It helps group data and related behavior together.

Class: A blueprint used to create objects. **Object:** An instance of a class. **Property/attribute:** Data stored in an object. **Method:** A function defined inside a class.

Example: Class and Object

```
class Student:
    name = "Shweta"
    age = 21

student1 = Student()
print(student1.name)
print(student1.age)
```

Output:

```
Shweta
21
```

2. Student Class with Properties and Methods

```
class Student:
    def __init__(self, name, roll_no, marks):
        self.name = name
        self.roll_no = roll_no
        self.marks = marks

    def display(self):
        print("Student Name:", self.name)
        print("Roll Number:", self.roll_no)
        print("Marks:", self.marks)

student1 = Student("Shweta", 72, 85)
student1.display()
```

Output:

```
Student Name: Shweta
Roll Number: 72
Marks: 85
```

Explanation: `__init__()` initializes object data and is called when an object is created. `self` refers to the current object. `display()` prints its information.

3. Constructors Using __init__()

A constructor initializes an object's data when it is created. In Python, `__init__()` is commonly used for this purpose.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def display(self):
        print("Employee Name:", self.name)
        print("Salary:", self.salary)

emp1 = Employee("Rahul", 25000)
emp2 = Employee("Priya", 30000)
emp1.display()
emp2.display()
```

Output:

```
Employee Name: Rahul
Salary: 25000
```

```
Employee Name: Priya
Salary: 30000
```

4. Encapsulation

Encapsulation groups data and related methods in a class. In Python, a name beginning with double underscores, such as `__marks`, uses name mangling to discourage direct access; it is not strict private access control.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.__marks = marks

    def display(self):
        print("Name:", self.name)
        print("Marks:", self.__marks)

    def update_marks(self, marks):
        if 0 <= marks <= 100:
            self.__marks = marks
        else:
            print("Invalid marks")

s1 = Student("Shweta", 85)
s1.display()
s1.update_marks(90)
s1.display()
```

Output:

```
Name: Shweta
Marks: 85
Name: Shweta
Marks: 90
```

5. Inheritance

Inheritance allows a child class to reuse or extend functionality from a parent class. `super()` can call a parent class method, including its constructor.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display_person(self):
        print("Name:", self.name)
        print("Age:", self.age)

class Student(Person):
    def __init__(self, name, age, roll_no):
        super().__init__(name, age)
        self.roll_no = roll_no

    def display_student(self):
        self.display_person()
        print("Roll Number:", self.roll_no)

s1 = Student("Shweta", 21, 72)
s1.display_student()
```

Output:

```
Name: Shweta
Age: 21
Roll Number: 72
```

6. Polymorphism

Polymorphism allows the same method name to behave differently for different objects.

```
class Student:
    def role(self):
        print("I am a student")

class Employee:
    def role(self):
        print("I am an employee")

person1 = Student()
person2 = Employee()
person1.role()
person2.role()
```

Output:

```
I am a student
I am an employee
```

7. Method Overriding

Method overriding occurs when a child class defines a method with the same name as a method in its parent class.

```
class Person:
    def display(self):
        print("This is a person")

class Student(Person):
    def display(self):
        print("This is a student")

s1 = Student()
s1.display()
```

Output:

```
This is a student
```

8. Abstraction

Abstraction exposes the required interface while hiding implementation details. Python supports abstract classes through the abc module.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def calculate_area(self):
        pass

class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def calculate_area(self):
        return self.length * self.width

r1 = Rectangle(10, 5)
print("Area:", r1.calculate_area())
```

Output:

```
Area: 50
```

9. Access Conventions in Python

Public: `name` — normally accessible from outside the class.
Internal-use convention: `_name` — indicates intended internal use.
Name mangling: `__name` — Python changes the attribute name internally to reduce accidental clashes and discourage direct access.

These conventions do not provide the same strict access control as private/protected modifiers in some other languages.

10. Student Management Console Application

This menu-driven program can add, view, search, update marks, and delete student records. It uses classes, objects, constructors, methods, and encapsulation. Records are stored in memory and are cleared when the program exits.

Create the file

In your Python_Practice folder, open PowerShell and run:

```
notepad student_management.py
```

Paste the complete code below into Notepad. Do not include Markdown code fences. Save with Ctrl+S, then close Notepad.

```
class Student:
    def __init__(self, roll_no, name, age, marks):
        self.roll_no = roll_no
        self.name = name
        self.age = age
        self.__marks = marks

    def get_marks(self):
        return self.__marks

    def update_marks(self, marks):
```

```
        if 0 <= marks <= 100:
            self.__marks = marks
            return True
        return False

    def display(self):
        print("Roll Number:", self.roll_no)
        print("Name:", self.name)
        print("Age:", self.age)
        print("Marks:", self.__marks)
        print("-" * 25)


class StudentManagement:
    def __init__(self):
        self.students = {}

    def add_student(self):
        roll_no = input("Enter roll number: ").strip()
        if not roll_no:
            print("Roll number cannot be empty.")
            return
        if roll_no in self.students:
            print("Student already exists.")
            return

        name = input("Enter student name: ").strip()
        if not name:
            print("Student name cannot be empty.")
            return

        try:
            age = int(input("Enter age: "))
            marks = float(input("Enter marks (0-100): "))
            if age <= 0 or not 0 <= marks <= 100:
                print("Invalid age or marks.")
                return
        except ValueError:
            print("Please enter valid numeric values for age and marks.")
            return

        self.students[roll_no] = Student(roll_no, name, age, marks)
        print("Student added successfully!")

    def view_students(self):
        if not self.students:
            print("No students found.")
            return
        for student in self.students.values():
            student.display()

    def search_student(self):
        roll_no = input("Enter roll number to search: ").strip()
        student = self.students.get(roll_no)
        if student:
            student.display()
        else:
            print("Student not found.")

    def update_student(self):
        roll_no = input("Enter roll number to update: ").strip()
        student = self.students.get(roll_no)
        if not student:
            print("Student not found.")
            return
        try:
            marks = float(input("Enter new marks (0-100): "))
            if student.update_marks(marks):
                print("Marks updated successfully!")
            else:
                print("Marks must be between 0 and 100.")
        except ValueError:
            print("Please enter valid marks.")

    def delete_student(self):
        roll_no = input("Enter roll number to delete: ").strip()
        if roll_no in self.students:
            del self.students[roll_no]
            print("Student deleted successfully!")
        else:
            print("Student not found.")

    def run(self):
        while True:
            print("\n===== STUDENT MANAGEMENT SYSTEM =====")
            print("1. Add Student")
            print("2. View All Students")
            print("3. Search Student")
            print("4. Update Student Marks")
```

```
            print("5. Delete Student")
            print("0. Exit")

            choice = input("Enter your choice: ").strip()
            if choice == "1":
                self.add_student()
            elif choice == "2":
                self.view_students()
            elif choice == "3":
                self.search_student()
            elif choice == "4":
                self.update_student()
            elif choice == "5":
                self.delete_student()
            elif choice == "0":
                print("Thank you for using Student Management System!")
                break
            else:
                print("Invalid choice. Please select 0 to 5.")


if __name__ == "__main__":
    app = StudentManagement()
    app.run()
```

Run the application

```
py student_management.py
```

Example: Add a student

```
===== STUDENT MANAGEMENT SYSTEM =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student Marks
5. Delete Student
0. Exit
Enter your choice: 1
Enter roll number: 72
Enter student name: Shweta
Enter age: 21
Enter marks (0-100): 85
Student added successfully!
```

11. Practice Tasks

1. Create a Student object with your own name, roll number, and marks.
2. Create an Employee class with name, employee ID, and salary.
3. Add a method to display employee details.
4. Create a Person parent class and Student child class.
5. Override a method in a child class.
6. Test all menu options in the Student Management application.

12. Quick Revision

Class: Blueprint for objects.
Object: Instance of a class.
Constructor: `__init__()`, initializes object data.
Encapsulation: Bundles data and methods and helps control access by convention.
Inheritance: Reuses or extends a parent class.
Polymorphism: Same method/interface name, different behavior.
Abstraction: Defines an interface while hiding implementation details.
Method overriding: Child class replaces inherited method behavior.